COL100: Introduction to Computer Science
Lab 10: Writing Better Code

March 24, 2025

1 Python Style Guidelines

PEP 8 is the official style guide for Python code, created by Guido van Rossum, Barry Warsaw, and Nick Coghlan. It provides coding conventions that enhance readability and consistency in Python code. Following PEP 8 is considered best practice in the Python community.

1.1 Code Layout

1.1.1 Indentation

- Use 4 spaces per indentation level
- Continuation lines should align wrapped elements vertically or use a hanging indent of 4 spaces
- The closing brace/bracket/parenthesis on multiline constructs should line up with the first character of the line that starts the construct

```
# Good - vertical alignment
my_list = [
    1, 2, 3,
    4, 5, 6,
]
# Good - hanging indent
foo = long_function_name(
    var_one, var_two,
    var_three, var_four)

# Bad - mixing tabs and spaces
def mixed_indentation():
  first_line_with_tab()  # Tab character here
    second_line_with_spaces()  # Spaces here
```

1.1.2 Maximum Line Length

- Limit all lines to a maximum of 79 characters for code
- Limit docstrings/comments to 72 characters

- Use parentheses for implicit line continuation
- In comments with large URLs, the URL may extend beyond the line limit

```
# Good - breaking long lines
income = (gross_wages
          + taxable_interest
          + (dividends - qualified_dividends)
          - ira_deduction
          - student_loan_interest)

# Bad - exceeding line length
income = gross_wages + taxable_interest + (dividends -
    qualified_dividends) - ira_deduction - student_loan_interest
```

1.1.3 Blank Lines

- Surround top-level function and class definitions with two blank lines
- Method definitions inside a class are surrounded by a single blank line
- Use blank lines to separate logical sections in functions

```
# Good - proper blank line usage
class ClassOne:
    def method_one(self):
        print("Method one")

    def method_two(self):
        print("Method two")


class ClassTwo:
    def another_method(self):
        print("Another method")
```

1.1.4 Imports

- Imports should be on separate lines
- Imports are grouped in the following order, separated by a blank line:
  - Standard library imports
  - Related third-party imports
  - Local application/library specific imports
- Absolute imports are recommended over relative imports

- Wildcard imports (`from module import *`) should be avoided

```
# Good
import os
import sys

from subprocess import Popen, PIPE

from mymodule import MyClass
from mypackage.mymodule import MyOtherClass

# Bad
import os, sys
from subprocess import *
```

1.2 String Quotes

- Be consistent in your use of single or double quotes for strings
- When a string contains single or double quote characters, use the other one to avoid backslashes
- Triple-quoted strings use double quotes

```
# Good
single_quoted = 'This is a string with "double quotes" inside'
double_quoted = "This is a string with 'single quotes' inside"
triple_quoted = """This is a triple-quoted string"""

# Avoid
escaped = 'This string has \'single quotes\' that need escaping'
```

1.3 Whitespace

1.3.1 Around Operators

- Surround binary operators with a single space on either side
- Don't use spaces around the '=' sign when used to indicate a keyword argument or default parameter value

```
# Good
x = 1
y = 2
long_variable = 3
i += 1
submitted += 1
```

```
x = x * 2 - 1
hypot2 = x*x + y*y
c = (a+b) * (a-b)
def complex(real, imag=0.0): return Complex(real, imag)

# Bad
x=1
y = 2  ; z = 3
i=i+1
submitted +=1
x = x*2 - 1
hypot2 = x * x + y * y
c = (a + b) * (a - b)
def complex(real, imag = 0.0): return Complex(real, imag)
```

1.3.2 Inside Parentheses/Brackets/Braces

- Avoid extraneous whitespace inside parentheses, brackets, or braces

```
% # Good
% spam(ham[1], {eggs: 2})

% # Bad
% spam( ham[ 1 ], { eggs: 2 } )
```

1.3.3 Around Commas, Colons, and Semicolons

- No space before a comma, semicolon, or colon
- Space after a comma, semicolon, or colon except at the end of a line

```
# Good
if x == 4: print(x, y); x, y = y, x

# Bad
if x == 4 : print(x , y) ; x , y = y , x
```

1.3.4 Before Parentheses

No space before the opening parenthesis that starts an argument list, indexing or slicing

```
# Good
spam(1)
dict['key'] = list[index]

# Bad
spam (1)
dict ['key'] = list [index]
```

1.3.5 Around Function Annotations

Use spaces around the '– >' arrow

```
# Good
def munge(input: AnyStr) -> AnyStr: ...
def munge() -> None: ...

# Bad
def munge(input:AnyStr)->PosInt: ...
```

1.4 Naming Conventions

1.4.1 General

Never use l, O, or I as single-character variable names (too easily confused with 1 and 0)

1.4.2 Functions and Variables

- Function names should be lowercase, with words separated by underscores (snake_case)
- Variable names follow the same convention as functions

```
def calculate_average(first_value, second_value):
    total_sum = first_value + second_value
    return total_sum / 2
```

1.4.3 Classes

Class names should use the CapWords convention (also known as PascalCase). We will be covering classes in the next lab.

```
class UserAccount:
    pass

class ValidParenthesis:
    pass
```

1.4.4 Constants

Constants are usually defined at the module level and written in all capital letters with underscores separating words.

```
MAX_OVERFLOW = 999
TOTAL = 0
```

1.5 Programming Recommendations

1.5.1 Comparisons

- Use `is` or `is not` when comparing with `None`
- Use explicit comparisons for boolean conditions

```
# Good
if x is not None:
    # Do something

# Good - explicit boolean check
if is_valid:
    # Do something

# Bad
if x:
    # This might be ambiguous if x could be None, False, 0, empty
    string, etc.
```

1.5.2 Exception Handling

- Be specific about which exceptions to catch
- Use the `try/except/else/finally` pattern appropriately

```
# Good - specific exception handling
try:
    value = mapping[key]
except KeyError:
    return default_value
else:
    return process_value(value)

# Bad - catching all exceptions
try:
    # Some code
except:  # This is too broad
    # Handle any exception
```

1.5.3 Return Statements

- Be consistent in return statements
- Either all return statements in a function should return an expression, or none of them should

```python
# Good
def foo(x):
    if x >= 0:
        return math.sqrt(x)
    else:
        return None

# Better
def foo(x):
    if x >= 0:
        return math.sqrt(x)
    return None
```

1.6 Other PEPs Related to Style

1.6.1 PEP 257 - Docstring Conventions

Provides guidelines for writing docstrings in Python.

```python
def calculate_area(width, height):
    """Calculate the area of a rectangle.

    Args:
        width (float): The width of the rectangle.
        height (float): The height of the rectangle.

    Returns:
        float: The calculated area.
    """
    return width * height
```

1.6.2 PEP 484 - Type Hints

Introduces a standard syntax for type annotations in Python.

```python
def greeting(name: str) -> str:
    return 'Hello ' + name
```

1.6.3 PEP 526 - Variable Annotations

Defines syntax for variable annotations.

```python
name: str = "John Doe"
age: int = 30
```

1.7 Tools to Enforce PEP 8

- **Pylint** A comprehensive linter that checks for errors and enforces coding standards

- **Flake8** Combines PyFlakes, pycodestyle (formerly pep8), and McCabe complexity checker
- **Black** An opinionated code formatter that automatically formats code to comply with PEP 8
- **YAPF** Yet Another Python Formatter from Google
- **autopep8** Automatically formats Python code to conform to the PEP 8 style guide

1.7.1 IDE Integration

Most modern Python IDEs include built-in support for PEPs:

- **PyCharm** Has built-in PEP 8 checking
- **VS Code** Can use the Python extension with various linting options
- **Jupyter Notebooks** Can use nbextensions for linting

1.8 Conclusion

Following PEP 8 guidelines makes your Python code more readable and maintainable. However, as PEP 8 itself states: "A style guide is about consistency. Consistency within a project is more important than personal style preferences." In cases where PEP 8 conflicts with readability, prioritize readability, and be consistent in your approach.

1.9 Example Code (Project Euler 26)

A unit fraction contains in the numerator. The decimal representation of the unit fractions with denominators 2 to 10 are given:

```
1/2 = 0.5
1/3 = 0. (3)
1/4 = 0.25 1/5 = 0.2
1/6 = 0.1 (6)
1/7 = 0. (142857)
1/8 = 0.125
1/9 = 0. (1)
1/10 = 0.1
```

Where 0.1 (6) means 0.16666—, and has a 1-digit recurring cycle. It can be seen that 1/7 has a 6-digit recurring cycle.
Find the value of $d < 1000$ for which $1/d$ contains the longest recurring cycle in its decimal fraction part.

Solution

```python
def calculate_cycle_length(d):
    """
    Calculate the length of the recurring cycle in the decimal
    representation of 1/d using long division.

    Args:
        d (int): The denominator

    Returns:
        int: The length of the recurring cycle (0 if terminating)
    """
    # Numbers that are divisible by 2 or 5 have terminating or
    # short cycles, handle separately for efficiency
    if d % 2 == 0 or d % 5 == 0:
        # Still calculate the actual cycle length for these numbers
        return calculate_cycle_length_with_long_division(d)

    return calculate_cycle_length_with_long_division(d)


def calculate_cycle_length_with_long_division(d):
    """
    Perform long division to find the recurring cycle length.

    Args:
        d (int): The denominator

    Returns:
        int: The length of the recurring cycle
    """
    # Dictionary to store remainders and their positions
    # Key: remainder, Value: position where this remainder occurred
    remainders = {}

    # Start with numerator 1
    numerator = 1
    position = 0

    while numerator != 0:
        # If numerator is less than denominator, multiply by 10 (shift
    decimal)
        numerator *= 10

        # Perform division step: quotient = numerator // d
        quotient = 0
        while numerator >= d:
            numerator -= d
            quotient += 1

```

```
        # The remainder is now in numerator
        remainder = numerator

        # If we've seen this remainder before, we've found a cycle
        if remainder in remainders:
            return position - remainders[remainder]

        # If remainder is 0, the division terminates
        if remainder == 0:
            return 0

        # Store the current remainder and its position
        remainders[remainder] = position
        position += 1

        # The remainder becomes our new numerator for the next step
        numerator = remainder

    # If we exit the loop, it's a terminating decimal
    return 0


def find_longest_recurring_cycle(limit):
    """
    Find the value of d < limit for which 1/d has the longest recurring
    cycle.

    Args:
        limit (int): The upper bound for d

    Returns:
        tuple: (d_with_max_cycle, max_cycle_length)
    """
    max_length = 0
    number_with_max_length = 0

    for d in range(2, limit):
        cycle_length = calculate_cycle_length(d)

        if cycle_length > max_length:
            max_length = cycle_length
            number_with_max_length = d

    return number_with_max_length, max_length


def main():
    """Main function to solve Project Euler Problem 26."""
    limit = 1000
    result, cycle_length = find_longest_recurring_cycle(limit)

```

```
    print(f"The value of d < {limit} with the longest recurring cycle is: {result}")
    print(f"The recurring cycle length is: {cycle_length}")

    # Optional: Display the decimal expansion to verify
    decimal_expansion = []
    numerator = 1
    remainders = {}
    cycle_start = None

    # Calculate first 100 digits or until cycle detected
    for i in range(100):
        numerator *= 10

        # Perform division without using / or //
        quotient = 0
        while numerator >= result:
            numerator -= result
            quotient += 1

        # If remainder repeats, we've detected the cycle
        if numerator in remainders and cycle_start is None:
            cycle_start = remainders[numerator]

        decimal_expansion.append(str(quotient))

        # If division terminates
        if numerator == 0:
            break

        remainders[numerator] = i

    # Format the output
    decimal_str = "0." + "".join(decimal_expansion)
    if cycle_start is not None:
        # Add parentheses around the recurring part
        non_recurring = "0." + "".join(decimal_expansion[:cycle_start])
        recurring = "".join(decimal_expansion[cycle_start:cycle_start + cycle_length])
        decimal_str = f"{non_recurring}({recurring})"

    print(f"1/{result} = {decimal_str}")


if __name__ == "__main__":
    main()
```

2 Submission Problem

A **round number** is a number that ends with one or more zeros in a given base.
Let us define the *roundness* of a number n in base b as the number of zeros at the end of the base b representation of n.
For example, 20 has roundness 2 in base 2, because the base 2 representation of 20 is 10100, which ends with 2 zeros.

Also define $R(n)$, the *total roundness* of a number n, as the sum of the roundness of n in base b for all $b > 1$. For example, 20 has roundness 2 in base 2 and roundness 1 in base 4, 5, 10, 20, hence we get $R(20) = 6$. You are also given $R(10!) = 312$. Give your answer modulo $10^9 + 7$.

Sample Input 1

The input consists of a expression. The expression maybe a number like Input 1 or a factorial like Input 2.

```
20
```

Sample Output 1

```
6
```

Sample Input 2

```
10!
```

Sample Output 2

```
312
```

Problem Reference: `https://projecteuler.net/problem=926`

3 Practice Problems

3.1 One Million Members

The number 6 can be written as a palindromic sum in exactly eight different ways:

$$(1,1,1,1,1,1),(1,1,2,1,1),(1,2,2,1),(1,4,1),(2,1,1,2),(2,2,2),(3,3),(6)$$

We shall define a twopal to be a palindromic tuple having at least one element with a value of 2. It should also be noted that elements are not restricted to single digits. For example, $(3,2,13,6,13,2,3)$ is a valid twopal.
If we let $t(n)$ be the number of twopals whose elements sum to n, then it can be seen that $t(6) = 4$:

$$(1,1,2,1,1),(1,2,2,1),(2,1,1,2),(2,2,2)$$

Similarly, $t(20) = 824$.
In searching for the answer to the ultimate question of life, the universe, and everything, it can be verified that $t(42) = 1999923$, which happens to be the first value of $t(n)$ that exceeds one million.
However, your challenge to the "ultimatest" question of life, the universe, and everything is to find the least value of $n < 42$ such that $t(n)$ is divisible by one million.

3.2 Sum of Squares II

For a positive integer, n, define $g(n)$ to be the maximum perfect square that divides n. For example, $g(18) = 9$, $g(19) = 1$.
Also define

$$S(N) = \sum_{n=1}^{N} g(n)$$

For example, $S(10) = 24$ and $S(100) = 767$.
Find $S(10^{14})$. Give your answer modulo $1\,000\,000\,007$.

Problem Reference: `https://projecteuler.net/problem=710`